SIMATS ENGINEERING

ASSESSMENT TOOL 3 – ANALYTICAL PROBLEM SOLVING

Data Pre-processing and Visualization Using Python: Five Case Studies

---

Course Name	**Fundamentals of Data Science**
Course Code	DSA04
Course Outcome	CO2: Apply data pre-processing and visualization techniques using Python libraries for effective data handling (BL4)
Number of Questions	05
Total Marks	100
Student Name / Reg. No.	G Chandravardhan Reddy - 192425037

Code of Conduct

I G Chandravardhan Reddy - 192425037 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: G Chandravardhan Reddy

Introduction and Common Workflow

This report answers the five case studies of Analytical Problem Solving (Annexure A). Each case describes a real organisation with raw data and asks how Pandas, data pre-processing and visualization are used to turn that data into decisions. Although the domains differ (food delivery, banking, energy, e-learning and public transport).

The Three-Stage Workflow

- **Stage A – Data handling:** Pandas loads data into DataFrames and performs selection, filtering, sorting, grouping and aggregation.
- **Stage B – Pre-processing:** missing values, duplicates, invalid entries and outliers are detected and treated; categorical variables are encoded and numeric variables are scaled or binned where needed.
- **Stage C – Visualization:** Matplotlib and Seaborn transform the cleaned data into charts that reveal distributions, comparisons, trends and relationships.

Python Libraries Used

Library	Role in the analysis
Pandas	DataFrames, reading files, cleaning, filtering, grouping, pivoting, date handling
NumPy	Numerical operations, conditional logic with np.select / np.where, NaN handling
Scikit-learn (preprocessing)	MinMaxScaler and StandardScaler for normalisation, encoders for categories
Matplotlib	Low-level control of figures: line, bar, histogram, scatter, subplots
Seaborn	Statistical charts: box, violin, count, heat map, pair plots with attractive defaults

Reusable Toolkit: Data Quality and Chart Selection

Standard Handling of Data Quality Issues

Issue	Typical detection	Typical remedy
Missing values	isnull().sum()	Drop, or impute with mean / median / mode, interpolation or group-wise values
Duplicates	duplicated()	drop_duplicates() with a suitable key
Outliers / invalid values	IQR rule, boxplot, domain limits	Remove, cap or replace after verification
Categorical text	unique(), value_counts()	Label encoding, one-hot encoding, mapping
Different scales	describe()	Min-max scaling or standardisation

Reusable Cleaning Pipeline

The following compact function captures the cleaning steps that recur across all five cases. It can be adapted by changing the key columns and limits.

```
def clean(df, key, num_cols, lo=None, hi=None):
    df = df.drop_duplicates(subset=key)              # remove duplicate records
    for c in num_cols:
        if lo is not None: df.loc[df[c] < lo, c] = None   # invalid low values
        if hi is not None: df.loc[df[c] > hi, c] = None   # invalid high values
        df[c] = df[c].fillna(df[c].median())          # median imputation
    return df.reset_index(drop=True)
```

Choosing the Right Chart

Analytical question	Recommended chart	Library call
How is one variable distributed?	Histogram, KDE, box plot	sns.histplot, sns.boxplot
How do categories compare?	Bar chart, count plot	sns.barplot, sns.countplot
How does a value change over time?	Line chart	df.plot(), plt.plot()
Are two numeric variables related?	Scatter plot	sns.scatterplot
Where are the hot spots in a grid?	Heat map	sns.heatmap
How do groups differ?	Grouped box or violin plot	sns.violinplot(hue=...)

1. Online Food Delivery Data Analysis

Scenario

An online food delivery company records every order with details such as order ID, restaurant, customer, order value, order time, delivery time and rating. Raw data of this kind is rarely analysis-ready: delivery times are sometimes missing, the same order may be logged twice, and prices may be mistyped. The workflow below shows how Pandas, preprocessing and visualization are combined to improve delivery performance and customer satisfaction.

Sample Dataset (Python)

The company's real data is not available, so a small synthetic dataset with the same kinds of defects is created below (random seed fixed for repeatability). All code in this section was executed and the results shown are the actual outputs.

Python code

```
import pandas as pd, numpy as np
pd.set_option('display.width', 90); pd.set_option('display.max_columns', 10)
np.random.seed(42)
n = 300
restaurants = ['Spice Hub','Pizza Point','Burger Barn','Dosa Corner','Biryani House','Wok
Express']
df = pd.DataFrame({
    'order_id': range(1001, 1001 + n),
    'restaurant': np.random.choice(restaurants, n),
    'city': np.random.choice(['Chennai','Bangalore'], n),
    'price': np.round(np.random.normal(350, 90, n), 2),
    'delivery_min': np.round(np.random.gamma(9, 4, n), 1),
    'rating': np.random.choice([1,2,3,4,5], n, p=[.05,.08,.17,.4,.3])})
# introduce typical data-quality problems
df.loc[np.random.choice(n, 20, replace=False), 'delivery_min'] = np.nan   # missing
delivery time
df.loc[[5, 17, 40], 'price'] = [-250, 0, 99999]                           # incorrect prices
df = pd.concat([df, df.sample(15, random_state=1)], ignore_index=True)    # duplicate
orders
df['restaurant'] = ' ' + df['restaurant'].str.lower() + ' '               # untidy text
print(df.head())
print('Shape:', df.shape)
```

Output

```
   order_id      restaurant       city   price  delivery_min  rating
0      1001     dosa corner   Bangalore  455.52          31.5       4
1      1002   biryani house   Bangalore  397.83          42.3       4
2      1003     burger barn     Chennai  417.42          60.0       4
3      1004   biryani house     Chennai  512.81          30.2       4
4      1005   biryani house     Chennai  370.43          13.9       5
Shape: (315, 6)
```

The sample dataset has 300 genuine orders plus 15 duplicated rows, 20 missing delivery times, three impossible prices and untidy restaurant names, so it reproduces the problems described in the question.

A. Use of Pandas

Pandas reads the order file into a DataFrame, the central two-dimensional table structure used for analysis. Functions such as read_csv() and read_excel() load the data, while head(), info(), describe() and isnull().sum() give a quick profile of column types, ranges and missing values. Cleaning includes fixing data types (for example converting order and delivery timestamps with pd.to_datetime()), trimming text, and standardising restaurant names. Filtering uses Boolean masks or query() to isolate the records of interest.

- **Read:** df = pd.read_csv('orders.csv') loads all orders into a DataFrame.
- **Inspect:** df.info() and df.describe() reveal dtypes, counts and suspicious minimum or maximum values.
- **Clean:** str.strip() and str.title() standardise restaurant names; astype() corrects numeric columns.
- **Filter:** df[df['delivery_time'] > 45] finds late orders; df[df['city']=='Chennai'] restricts the study to one city.

Python code

```
df.info()
print()
print(df.isnull().sum())
df['restaurant'] = df['restaurant'].str.strip().str.title()      # clean text
late = df[df['delivery_min'] > 45]                               # filter: delayed orders
print('\nLate orders (> 45 min):', len(late))
print(late.head(3))
good = df[(df['rating'] >= 4) & (df['city'] == 'Chennai')]
print('Chennai orders rated 4 or above:', len(good))
```

Output

```
<class 'pandas.DataFrame'>
RangeIndex: 315 entries, 0 to 314
Data columns (total 6 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   order_id      315 non-null    int64
 1   restaurant    315 non-null    str
 2   city          315 non-null    str
 3   price         315 non-null    float64
 4   delivery_min  294 non-null    float64
 5   rating        315 non-null    int64
dtypes: float64(2), int64(2), str(2)
```

```
memory usage: 14.9 KB

order_id        0
restaurant      0
city            0
price           0
delivery_min    21
rating          0
dtype: int64

Late orders (> 45 min): 77
    order_id     restaurant       city   price  delivery_min  rating
2       1003    Burger Barn    Chennai  417.42          60.0       4
9       1010  Biryani House  Bangalore  445.46          54.4       4
13      1014  Biryani House  Bangalore  235.94          56.8       3
Chennai orders rated 4 or above: 94
```

The profile shows 315 rows with 21 missing delivery times. After text clean-up, boolean filtering finds 77 orders that took longer than 45 minutes, and 94 Chennai orders have a rating of 4 or above.

B. Data Preprocessing

Data preprocessing converts messy records into reliable data. Each of the three issues named in the problem has a different remedy, chosen according to how the error arises and how much it can distort the results.

Problem	Detection	Treatment
Missing delivery time	df.isnull().sum()	Impute with the median delivery time of the same restaurant or area (fillna / groupby-transform); drop rows if the order was cancelled.
Duplicate orders	df.duplicated(subset='order_id')	drop_duplicates(subset='order_id', keep='first') keeps one record per order.
Incorrect prices	describe(), boxplot, price <= 0	Treat zero, negative or extreme values as errors; replace with NaN and impute with the item median, or remove them.

Python code

```
print('Duplicate orders before :', df.duplicated(subset='order_id').sum())
df = df.drop_duplicates(subset='order_id', keep='first')
print('Duplicate orders after  :', df.duplicated(subset='order_id').sum())

bad_price = (df['price'] <= 0) | (df['price'] > 5000)
print('Incorrect prices found  :', bad_price.sum())
df.loc[bad_price, 'price'] = np.nan
df['price'] = df['price'].fillna(df['price'].median())
```

```
df['delivery_min'] = df.groupby('restaurant')['delivery_min'].transform(
    lambda s: s.fillna(s.median()))
print('\nMissing values after cleaning:')
print(df.isnull().sum())
print('\n', df[['price', 'delivery_min']].describe().round(2))
```

Output

```
Duplicate orders before : 15
Duplicate orders after  : 0
Incorrect prices found  : 3

Missing values after cleaning:
order_id        0
restaurant      0
city            0
price           0
delivery_min    0
rating          0
dtype: int64

         price  delivery_min
count  300.00        300.00
mean   356.12         37.37
std     95.70         12.28
min     36.96         11.50
25%    293.19         29.35
50%    359.89         35.70
75%    409.14         44.32
max    642.86         87.10
```

All 15 duplicate orders and the 3 incorrect prices were handled, and delivery times were filled with each restaurant's own median. The final table has 300 rows and no missing values.

C. Visualization

Visualization turns the cleaned numbers into patterns that managers can act upon. Matplotlib provides full control over figures, while Seaborn offers statistical charts with attractive defaults.

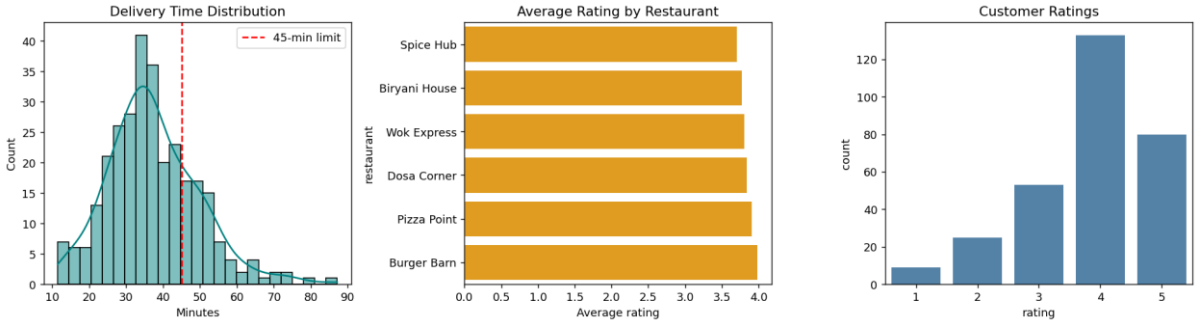
- **Histogram or box plot of delivery_min –** shows the spread of delivery delays and outliers; a vertical line at 45 minutes marks the service-level limit.
- **Horizontal bar chart of top 10 restaurants –** ranks restaurants by order count or average rating using value_counts() or groupby().mean().
- **Count plot or violin plot of ratings –** shows how customer ratings are distributed and whether late deliveries receive lower scores.

- **Heatmap of hour versus weekday delays** – reveals peak-hour congestion for rider planning.

Python code

```
import seaborn as sns
import matplotlib.pyplot as plt
fig, ax = plt.subplots(1, 3, figsize=(15, 4.2))
sns.histplot(df['delivery_min'], bins=25, kde=True, color='teal', ax=ax[0])
ax[0].axvline(45, color='red', ls='--', label='45-min limit'); ax[0].legend()
ax[0].set_title('Delivery Time Distribution'); ax[0].set_xlabel('Minutes')
top = df.groupby('restaurant')['rating'].mean().sort_values()
sns.barplot(x=top.values, y=top.index, color='orange', ax=ax[1])
ax[1].set_title('Average Rating by Restaurant'); ax[1].set_xlabel('Average rating')
sns.countplot(x='rating', data=df, color='steelblue', ax=ax[2])
ax[2].set_title('Customer Ratings')
plt.tight_layout(); plt.show()
print(df.groupby('restaurant').agg(orders=('order_id','count'),
    avg_delay=('delivery_min','mean'), avg_rating=('rating','mean')).round(2)
    .sort_values('orders', ascending=False))
```

Output – chart



Output – printed results

restaurant	orders	avg_delay	avg_rating
Dosa Corner	64	39.79	3.84
Spice Hub	57	38.69	3.70
Burger Barn	52	34.39	3.98
Wok Express	46	38.99	3.80
Pizza Point	42	35.19	3.90
Biryani House	39	35.84	3.77

Insight and Business Value

The analysis lets the company identify restaurants and time slots with chronic delays, reward high-performing partners and investigate the causes of low ratings. Clean, de-duplicated data ensures that revenue and delay figures are not overstated.

2. Bank Loan Approval Analysis

Scenario

A bank holds thousands of loan applications containing applicant income, loan amount, credit score, gender, employment type and the final decision. The aim is to discover which characteristics are associated with approval or rejection. Because the data mixes numbers and categories and contains gaps, careful DataFrame handling and preprocessing are required before any pattern can be trusted.

Sample Dataset (Python)

The company's real data is not available, so a small synthetic dataset with the same kinds of defects is created below (random seed fixed for repeatability). All code in this section was executed and the results shown are the actual outputs.

Python code

```
import pandas as pd, numpy as np
pd.set_option('display.width', 90); pd.set_option('display.max_columns', 12)
np.random.seed(7)
n = 400
income = np.round(np.random.lognormal(10.9, 0.45, n), -2)
credit = np.clip(np.random.normal(690, 70, n), 400, 850).round()
loan = np.round(np.random.normal(250000, 90000, n).clip(30000, 600000), -3)
score = (credit - 650) / 70 + (income - 60000) / 40000 - (loan / income) * 0.6 +
np.random.normal(0, .6, n)
df = pd.DataFrame({'gender': np.random.choice(['Male', 'Female'], n),
                   'employment': np.random.choice(['Salaried', 'Self-Employed', 'Business'], n,
p=[.55, .25, .2]),
                   'income': income, 'loan_amount': loan, 'credit_score': credit,
                   'status': np.where(score > -1.8, 'Approved', 'Rejected')})
df.loc[np.random.choice(n, 30, replace=False), 'income'] = np.nan      # missing income
print(df.head())
print('\nShape:', df.shape)
print(df['status'].value_counts())
```

Output

```
   gender     employment    income  loan_amount  credit_score    status
0  Female  Self-Employed  115900.0     259000.0         686.0  Approved
1    Male       Business   43900.0     208000.0         745.0  Rejected
2    Male  Self-Employed   55000.0     114000.0         590.0  Rejected
3    Male       Salaried   65100.0     289000.0         720.0  Approved
4    Male       Salaried   38000.0     271000.0         700.0  Rejected

Shape: (400, 6)
status
Rejected    245
Approved    155
```

```
Name: count, dtype: int64
```

The dataset has 400 applications (155 approved, 245 rejected) with 30 missing income values, which are filled in the preprocessing step.

A. Use of Pandas

Pandas DataFrame operations make it easy to focus on the columns that matter. Column selection with double brackets creates a smaller DataFrame, while loc and iloc allow label-based or position-based selection of rows and columns. Boolean conditions narrow the data to specific groups, and groupby() produces summaries such as the mean income of approved versus rejected applicants.

- **Select columns:** df[['income','loan_amount','credit_score','status']] keeps only the analysis variables.
- **Label-based access:** df.loc[df['credit_score']>700, ['income','status']] selects chosen rows and columns.
- **Group summary:** df.groupby('status')[['income','credit_score','loan_amount']].mean() compares the two outcomes.
- **Cross-tabulation:** pd.crosstab(df['employment'], df['status'], normalize='index') shows approval rate by employment type.

Python code

```
sel = df[['income', 'loan_amount', 'credit_score', 'status']]       # column selection
print(sel.head())
print('\nAverage values by approval status:')
print(sel.groupby('status').mean().round(0))
strong = df.loc[(df['credit_score'] >= 750) & (df['income'] > 60000), ['income',
'credit_score', 'status']]
print('\nApplicants with score >= 750 and income > 60000:', len(strong))
print(strong['status'].value_counts())
print('\nApproval rate by employment type:')
print(pd.crosstab(df['employment'], df['status'], normalize='index').round(2))
```

Output

```
    income  loan_amount  credit_score    status
0  115900.0     259000.0         686.0  Approved
1   43900.0     208000.0         745.0  Rejected
2   55000.0     114000.0         590.0  Rejected
3   65100.0     289000.0         720.0  Approved
4   38000.0     271000.0         700.0  Rejected

Average values by approval status:
          income  loan_amount  credit_score
status
Approved  79199.0     209981.0         719.0
```

```
Rejected  45516.0     279127.0         664.0

Applicants with score >= 750 and income > 60000: 31
status
Approved    31
Name: count, dtype: int64

Approval rate by employment type:
status         Approved  Rejected
employment
Business           0.39      0.61
Salaried           0.38      0.62
Self-Employed      0.41      0.59
```

Approved applicants have a clearly higher average income (79,199 against 45,516) and credit score (719 against 664), and request smaller loans (209,981 against 279,127). All 31 applicants with a score of at least 750 and an income above 60,000 were approved.

B. Data Preprocessing

Machine-learning models and statistical comparisons need complete, numeric and comparably scaled inputs. The three preprocessing needs in this case are handled as follows.

Task	Technique	Pandas / Scikit-learn method
Missing income	Imputation	fillna(df['income'].median()) – median is preferred because income is right-skewed; group-wise medians by employment type can refine this.
Encode gender and employment type	Label or one-hot encoding	map({'Male':0,'Female':1}) for binary gender; pd.get_dummies(columns=['employment']) for multi-category variables.
Normalize loan amount	Min-max scaling or standardisation	MinMaxScaler gives values from 0 to 1; StandardScaler gives zero mean and unit variance.

Python code

```
from sklearn.preprocessing import MinMaxScaler
print('Missing income before:', df['income'].isnull().sum())
df['income'] = df['income'].fillna(df.groupby('employment')['income'].transform('median'))
print('Missing income after :', df['income'].isnull().sum())

df['gender'] = df['gender'].map({'Male': 0, 'Female': 1})          # label encoding
df = pd.get_dummies(df, columns=['employment'], drop_first=True, dtype=int)   # one-hot
encoding
df['loan_scaled'] = MinMaxScaler().fit_transform(df[['loan_amount']]).round(3)  #
normalisation
print()
```

```
print(df.head())
print('\nScaled loan range:', df['loan_scaled'].min(), 'to', df['loan_scaled'].max())
```

Output

```
Missing income before: 30
Missing income after : 0

   gender    income  loan_amount  credit_score    status  employment_Salaried  \
0       1  115900.0     259000.0         686.0  Approved                    0
1       0   43900.0     208000.0         745.0  Rejected                    0
2       0   55000.0     114000.0         590.0  Rejected                    0
3       0   65100.0     289000.0         720.0  Approved                    1
4       0   38000.0     271000.0         700.0  Rejected                    1

   employment_Self-Employed  loan_scaled
0                         1        0.504
1                         0        0.392
2                         1        0.185
3                         0        0.570
4                         0        0.531

Scaled loan range: 0.0 to 1.0
```

The 30 missing incomes were replaced with the median income of the same employment type. Gender is now 0/1, employment is split into indicator columns, and loan_scaled lies between 0 and 1.

C. Visualization

Visual comparison of the two groups makes patterns immediately visible and communicates findings to non-technical stakeholders such as credit managers.

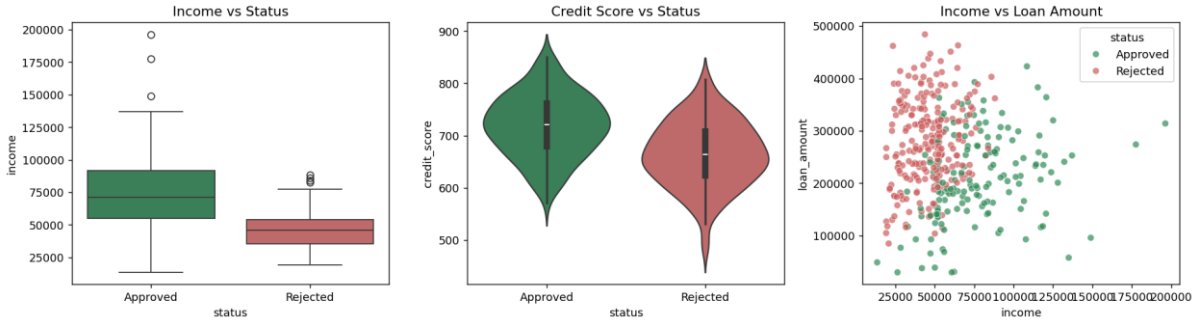
- **Box plot of income by status –** compares the median and spread of income for approved and rejected applicants.
- **Violin or KDE plot of credit score by status –** shows how strongly higher scores concentrate in the approved group.
- **Scatter plot of income versus loan amount (hue = status) –** reveals the loan-to-income ratio beyond which rejection becomes common.
- **Bar chart of approval rate by employment type –** highlights segments with higher risk.

Python code

```
import seaborn as sns
import matplotlib.pyplot as plt
fig, ax = plt.subplots(1, 3, figsize=(15, 4.2))
pal = {'Approved': 'seagreen', 'Rejected': 'indianred'}
sns.boxplot(x='status', y='income', data=df, palette=pal, ax=ax[0]); ax[0].set_title('Income
vs Status')
```

```
sns.violinplot(x='status', y='credit_score', data=df, palette=pal, ax=ax[1]);
ax[1].set_title('Credit Score vs Status')
sns.scatterplot(x='income', y='loan_amount', hue='status', palette=pal, data=df, alpha=.7,
ax=ax[2])
ax[2].set_title('Income vs Loan Amount')
plt.tight_layout(); plt.show()
print(df.groupby('status')[['income', 'credit_score', 'loan_amount']].median())
```

Output – chart



Output – printed results

```
          income  credit_score  loan_amount
status
Approved  71100.0         721.0     206000.0
Rejected  46000.0         664.0     277000.0
```

The box plot, violin plot and scatter plot show the same story: approved applications cluster at higher income and credit score and at lower loan-to-income ratios, while rejected ones dominate the area of low income with large loans.

Insight and Business Value

Typically, approved applicants show higher credit scores and incomes and lower loan-to-income ratios. These findings can guide credit policy, but they must be reviewed for fairness, particularly when gender or other sensitive attributes are involved.

3. Smart Energy Consumption Monitoring

Scenario

A smart home system logs electricity consumption (in kWh) for appliances such as the AC, fan, light and refrigerator. Sensors can fail or spike, producing missing or abnormal readings. The goal is to compute reliable consumption totals, identify the most energy-hungry appliances and understand daily usage trends.

Sample Dataset (Python)

The company's real data is not available, so a small synthetic dataset with the same kinds of defects is created below (random seed fixed for repeatability). All code in this section was executed and the results shown are the actual outputs.

Python code

```
import pandas as pd, numpy as np
pd.set_option('display.width', 90); pd.set_option('display.max_columns', 12)
np.random.seed(3)
dates = pd.date_range('2026-09-01', periods=30)
base = {'AC': 9.0, 'Fan': 1.2, 'Light': 0.8, 'Refrigerator': 2.0}
rows = [(d, a, max(0.1, np.random.normal(m, m * 0.12)))
        for d in dates for a, m in base.items()]
df = pd.DataFrame(rows, columns=['date', 'appliance', 'kwh']).round(2)
df.loc[[10, 45, 77], 'kwh'] = [48.0, 35.5, -3.0]              # abnormal sensor readings
df.loc[[20, 33, 61, 90, 102], 'kwh'] = np.nan                  # missing readings
print(df.head(8))
print('Shape:', df.shape)
```

Output

```
        date     appliance    kwh
0 2026-09-01            AC  10.93
1 2026-09-01           Fan   1.26
2 2026-09-01         Light   0.81
3 2026-09-01  Refrigerator   1.55
4 2026-09-02            AC   8.70
5 2026-09-02           Fan   1.15
6 2026-09-02         Light   0.79
7 2026-09-02  Refrigerator   1.85
Shape: (120, 3)
```

The data contains 30 days of readings for four appliances (120 rows). Three abnormal readings (48.0, 35.5 and -3.0 kWh) and five missing readings were inserted to mimic faulty sensors.

A. Use of Pandas

Grouping is the core Pandas operation here. The date column is converted to datetime and optionally set as the index. groupby() splits the data by appliance, by date, or by both, applies

an aggregation such as sum() or mean(), and combines the result into a new DataFrame. pivot_table() reshapes the output into a date-by-appliance grid that is convenient for plotting.

- **By appliance:** df.groupby('appliance')['kwh'].sum() gives total usage per appliance.
- **By date:** df.groupby('date')['kwh'].sum() gives total household usage per day.
- **By both:** df.groupby(['date','appliance'])['kwh'].sum().unstack() creates one column per appliance.
- **Resampling:** df.set_index('date').resample('W')['kwh'].sum() aggregates to weekly totals.

Python code

```
total_app = df.groupby('appliance')['kwh'].sum().sort_values(ascending=False)
print('Total kWh by appliance (raw data):')
print(total_app.round(2))
daily = df.groupby(['date', 'appliance'])['kwh'].sum(min_count=1).unstack()     # date x
appliance table
print('\nDaily usage by appliance:')
print(daily.head())
print('\nWeekly totals:')
print(df.set_index('date').resample('W')['kwh'].sum().round(2))
```

Output

```
Total kWh by appliance (raw data):
appliance
AC             256.00
Light           69.59
Fan             63.89
Refrigerator    59.38
Name: kwh, dtype: float64

Daily usage by appliance:
appliance     AC   Fan  Light  Refrigerator
date
2026-09-01  10.93  1.26   0.81          1.55
2026-09-02   8.70  1.15   0.79          1.85
2026-09-03   8.95  1.13  48.00          2.21
2026-09-04   9.95  1.45   0.80          1.90
2026-09-05   8.41  0.98   0.89          1.74

Weekly totals:
date
2026-09-06    117.62
2026-09-13    114.53
2026-09-20     89.58
2026-09-27     86.55
2026-10-04     40.58
Freq: W-SUN, Name: kwh, dtype: float64
```

The raw totals are misleading: the Light total (69.59 kWh) is inflated by one 48 kWh spike. The date-by-appliance table makes this visible on 2026-09-03, which is why abnormal readings must be removed first.

B. Data Preprocessing

Sensor data must be cleaned before totals are calculated; otherwise a single faulty spike can distort daily consumption and mislead the user.

Task	Technique	Method
Remove abnormal readings	IQR rule or domain limits	Compute Q1, Q3 and IQR; flag kwh below Q1-1.5×IQR or above Q3+1.5×IQR; also drop negative values or values exceeding the appliance's rated capacity.
Fill missing values	Interpolation or forward fill	Time-series data is best filled with interpolate(method='time') or ffill(); an appliance mean can be used for isolated gaps.
Total power consumption	Aggregation	Create total columns with sum(axis=1) on the pivoted table or groupby().sum().

Python code

```
g = df.groupby('appliance')['kwh']                    # IQR rule per appliance
q1 = g.transform(lambda s: s.quantile(0.25)); q3 = g.transform(lambda s: s.quantile(0.75))
iqr = q3 - q1
abnormal = (df['kwh'] < q1 - 3 * iqr) | (df['kwh'] > q3 + 3 * iqr) | (df['kwh'] < 0)
print('Abnormal readings found:', abnormal.sum())
print(df[abnormal])
df.loc[abnormal, 'kwh'] = np.nan                      # mark abnormal as missing

pivot = df.pivot_table(index='date', columns='appliance', values='kwh', aggfunc=lambda s:
s.sum(min_count=1), dropna=False)
print('\nMissing values before filling:', pivot.isnull().sum().sum())
pivot = pivot.interpolate(method='linear').bfill().ffill()        # fill gaps
print('Missing values after filling :', pivot.isnull().sum().sum())
pivot['total_kwh'] = pivot[['AC', 'Fan', 'Light', 'Refrigerator']].sum(axis=1)
print()
print(pivot.head().round(2))
```

Output

```
Abnormal readings found: 3
         date appliance   kwh
10 2026-09-03     Light  48.0
45 2026-09-12       Fan  35.5
77 2026-09-20       Fan  -3.0
```

```
Missing values before filling: 8
Missing values after filling : 0

appliance      AC   Fan  Light  Refrigerator  total_kwh
date
2026-09-01  10.93  1.26   0.81          1.55      14.55
2026-09-02   8.70  1.15   0.79          1.85      12.49
2026-09-03   8.95  1.13   0.80          2.21      13.08
2026-09-04   9.95  1.45   0.80          1.90      14.10
2026-09-05   8.41  0.98   0.89          1.74      12.02
```

The IQR rule found the three abnormal readings. Together with the original gaps, 8 values were missing; linear interpolation filled all of them, and total_kwh now gives the whole-house consumption per day.

C. Visualization

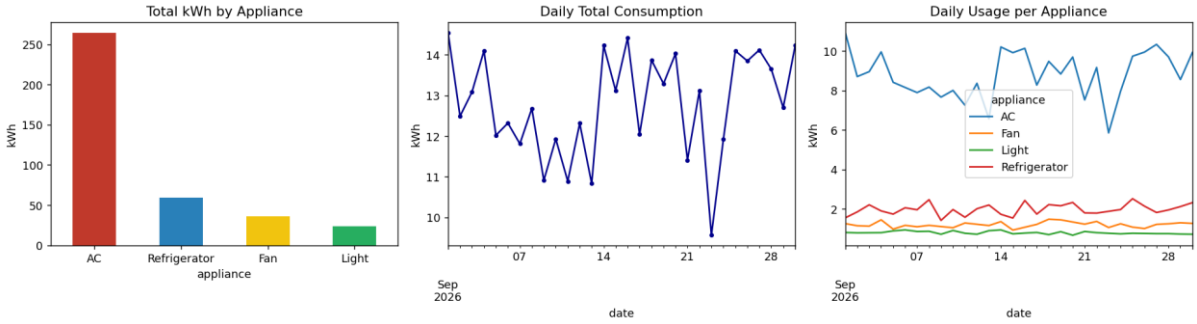
Time-based and category-based comparisons call for different chart types, so line and bar charts are used together.

- **Bar chart of total kWh per appliance –** immediately shows that high-power devices such as the AC dominate consumption.
- **Line chart of daily total kWh –** reveals trends, weekday and weekend differences and seasonal peaks.
- **Multi-line chart per appliance –** shows when each appliance contributes most, for example AC during afternoons.
- **Stacked bar chart per day –** displays each appliance's share of the daily total.

Python code

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots(1, 3, figsize=(15, 4.2))
totals = pivot[['AC', 'Fan', 'Light', 'Refrigerator']].sum().sort_values(ascending=False)
totals.plot(kind='bar', color=['#c0392b', '#2980b9', '#f1c40f', '#27ae60'], ax=ax[0])
ax[0].set_title('Total kWh by Appliance'); ax[0].set_ylabel('kWh');
ax[0].tick_params(axis='x', rotation=0)
pivot['total_kwh'].plot(ax=ax[1], marker='o', ms=3, color='darkblue')
ax[1].set_title('Daily Total Consumption'); ax[1].set_ylabel('kWh')
pivot[['AC', 'Fan', 'Light', 'Refrigerator']].plot(ax=ax[2])
ax[2].set_title('Daily Usage per Appliance'); ax[2].set_ylabel('kWh')
plt.tight_layout(); plt.show()
print('Total kWh by appliance:'); print(totals.round(2))
print('\nShare of consumption (%):'); print((totals / totals.sum() * 100).round(1))
print('\nPeak day:', pivot['total_kwh'].idxmax().date(), '-', round(pivot['total_kwh'].max(),
2), 'kWh')
```

Output – chart



Output – printed results

```
Total kWh by appliance:
appliance
AC              264.15
Refrigerator     59.38
Fan              36.14
Light            23.92
dtype: float64

Share of consumption (%):
appliance
AC              68.9
Refrigerator    15.5
Fan              9.4
Light            6.2
dtype: float64

Peak day: 2026-09-01 - 14.55 kWh
```

The AC consumes 264.15 kWh, about 68.9% of the total, followed by the refrigerator (15.5%), fan (9.4%) and lights (6.2%). The daily line chart shows a fairly stable demand with day-to-day variation driven mainly by the AC.

Insight and Business Value

The analysis typically identifies the air conditioner as the largest consumer and shows usage peaks on hot days. Residents can use these results to shift usage to off-peak hours, replace inefficient appliances and set consumption alerts.

4. Online Learning Platform Performance Analysis

Scenario

An online learning platform records each student's login time, video completion percentage, quiz score and assignment submission status. To support at-risk students and improve course design, the platform must rank learners, clean its activity logs and visualize engagement. The following steps describe how Pandas, preprocessing and Seaborn/Matplotlib support this analysis.

Sample Dataset (Python)

The company's real data is not available, so a small synthetic dataset with the same kinds of defects is created below (random seed fixed for repeatability). All code in this section was executed and the results shown are the actual outputs.

Python code

```
import pandas as pd, numpy as np
pd.set_option('display.width', 90); pd.set_option('display.max_columns', 12)
np.random.seed(11)
n = 200
completion = np.clip(np.random.normal(65, 22, n), 5, 100).round(1)
quiz = np.clip(completion * 0.6 + np.random.normal(20, 12, n), 0, 100).round(1)
d = pd.date_range('2026-08-01', periods=n, freq='6h')
login = [x.strftime('%d-%m-%Y %H:%M') if i % 2 == 0 else x.strftime('%Y/%m/%d
%H:%M')
        for i, x in enumerate(d)]                          # two different date formats
df = pd.DataFrame({'student_id': ['S' + str(100 + i) for i in range(n)],
                   'login_time': login, 'completion_pct': completion, 'quiz_score': quiz,
                   'assignment_submitted': np.random.choice(['Yes', 'No'], n, p=[.8, .2])})
df.loc[np.random.choice(n, 18, replace=False), 'quiz_score'] = np.nan   # missing quiz
scores
print(df.head())
print('Shape:', df.shape)
```

Output

```
  student_id       login_time  completion_pct  quiz_score assignment_submitted
0       S100  01-08-2026 00:00           100.0        NaN                 Yes
1       S101  2026/08/01 06:00            58.7       75.7                 Yes
2       S102  01-08-2026 12:00            54.3       33.5                 Yes
3       S103  2026/08/01 18:00             6.6       19.4                 Yes
4       S104  02-08-2026 00:00            64.8       72.8                 Yes
Shape: (200, 5)
```

The dataset has 200 students with login times stored in two different date formats and 18 missing quiz scores.

A. Use of Pandas

Pandas offers concise operations for ordering and selecting students. sort_values() orders records by one or more columns, boolean filtering selects groups such as students with low completion, and rank() assigns positions with options for handling ties. nlargest() and nsmallest() quickly return top or bottom performers.

- **Sort:** df.sort_values(['quiz_score','completion_pct'], ascending=False) orders students from best to weakest.
- **Filter:** df[(df['completion_pct']<50)&(df['quiz_score']<40)] flags students needing support.
- **Rank:** df['rank'] = df['quiz_score'].rank(ascending=False, method='dense') gives an explicit position.
- **Top performers:** df.nlargest(10,'quiz_score') returns the ten highest scorers.

Python code

```
ranked = df.sort_values(['quiz_score', 'completion_pct'], ascending=[False, False])
print('Top 5 students by quiz score:')
print(ranked.head(5)[['student_id', 'quiz_score', 'completion_pct']])
df['rank'] = df['quiz_score'].rank(ascending=False, method='dense')
at_risk = df[(df['completion_pct'] < 50) & (df['quiz_score'] < 40)]
print('\nAt-risk students (completion < 50 and score < 40):', len(at_risk))
print(at_risk[['student_id', 'quiz_score', 'completion_pct']].head())
print('\nStudents who did not submit the assignment:', (df['assignment_submitted'] ==
'No').sum())
```

Output

```
Top 5 students by quiz score:
    student_id  quiz_score  completion_pct
135       S235        98.9           98.9
181       S281        90.8           97.5
74        S174        90.2           94.6
128       S228        89.1          100.0
108       S208        86.9          100.0

At-risk students (completion < 50 and score < 40): 13
   student_id  quiz_score  completion_pct
3        S103        19.4            6.6
9        S109        31.3           41.6
14       S114        18.1           36.3
48       S148        26.1           27.9
79       S179        13.7           23.0

Students who did not submit the assignment: 43
```

Sorting and ranking identify the top performers (the best score is 98.9). Filtering finds 13 at-risk students with completion below 50% and a score below 40, and 43 students have not submitted the assignment.

B. Data Preprocessing

Preprocessing here covers missing scores, inconsistent date formats and the derivation of new, more meaningful features.

Task	Technique	Method
Missing quiz scores	Imputation or flagging	Fill with the course mean or median when the quiz was attempted; use 0 or a separate 'not attempted' flag if the quiz was skipped.
Convert date formats	Parsing	pd.to_datetime(df['login_time'], errors='coerce') unifies mixed formats; extract hour, weekday and month with the .dt accessor.
Create performance category	Feature engineering / binning	pd.cut() or np.select() converts scores into categories such as Excellent, Good, Average and Needs Improvement.

Python code

```
print('Missing quiz scores before:', df['quiz_score'].isnull().sum())
df['quiz_score'] = df['quiz_score'].fillna(df['quiz_score'].median())
print('Missing quiz scores after :', df['quiz_score'].isnull().sum())

d1 = pd.to_datetime(df['login_time'], format='%d-%m-%Y %H:%M', errors='coerce')   # format 1
d2 = pd.to_datetime(df['login_time'], format='%Y/%m/%d %H:%M', errors='coerce')   # format 2
df['login_time'] = d1.fillna(d2)                                                  # unified datetime
df['login_hour'] = df['login_time'].dt.hour
df['weekday'] = df['login_time'].dt.day_name()
df['performance'] = pd.cut(df['quiz_score'], bins=[-1, 40, 60, 80, 100],
                           labels=['Needs Improvement', 'Average', 'Good', 'Excellent'])
print('\nData type of login_time:', df['login_time'].dtype)
print(df[['student_id', 'login_time', 'login_hour', 'quiz_score', 'performance']].head())
print('\n', df['performance'].value_counts())
```

Output

```
Missing quiz scores before: 18
Missing quiz scores after : 0

Data type of login_time: datetime64[us]
  student_id         login_time  login_hour  quiz_score        performance
0       S100 2026-08-01 00:00:00           0        59.8            Average
1       S101 2026-08-01 06:00:00           6        75.7               Good
2       S102 2026-08-01 12:00:00          12        33.5  Needs Improvement
3       S103 2026-08-01 18:00:00          18        19.4  Needs Improvement
4       S104 2026-08-02 00:00:00           0        72.8               Good
```

```
 performance
Average              87
Good                 73
Needs Improvement    23
Excellent            17
Name: count, dtype: int64
```

The 18 missing scores were filled with the median, and both date formats were converted to a single datetime64 column. The new performance column places 87 students in Average, 73 in Good, 23 in Needs Improvement and 17 in Excellent.

C. Visualization

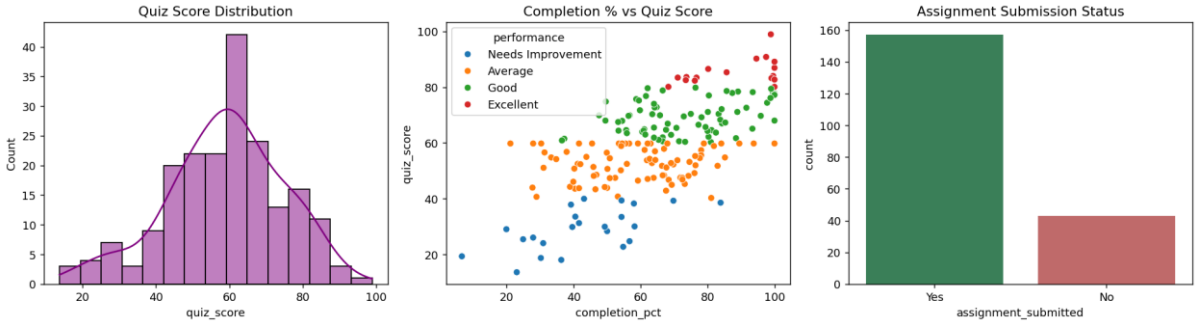
Visualization helps instructors see overall engagement and spot problems at a glance.

- **Histogram of quiz scores –** shows the score distribution and whether the quiz was too easy or too hard.
- **Scatter plot of completion % versus quiz score –** shows whether students who watch more video score higher.
- **Count plot of performance category –** summarises how many students fall in each band.
- **Pie or bar chart of completion status –** shows the proportion who completed, are in progress or dropped out.
- **Heatmap of login hour versus weekday –** reveals when students are most active.

Python code

```
import seaborn as sns
import matplotlib.pyplot as plt
fig, ax = plt.subplots(1, 3, figsize=(15, 4.2))
sns.histplot(df['quiz_score'], bins=15, kde=True, color='purple', ax=ax[0]);
ax[0].set_title('Quiz Score Distribution')
sns.scatterplot(x='completion_pct', y='quiz_score', hue='performance', data=df, ax=ax[1])
ax[1].set_title('Completion % vs Quiz Score')
sns.countplot(x='assignment_submitted', data=df, hue='assignment_submitted',
palette=['seagreen', 'indianred'], legend=False, ax=ax[2])
ax[2].set_title('Assignment Submission Status')
plt.tight_layout(); plt.show()
print('Correlation (completion vs score):', round(df['completion_pct'].corr(df['quiz_score']),
2))
print('\nAverage score by performance band:'); print(df.groupby('performance',
observed=True)['quiz_score'].mean().round(1))
```

Output – chart



Output – printed results

```
Correlation (completion vs score): 0.64

Average score by performance band:
performance
Needs Improvement    29.2
Average              52.6
Good                 69.3
Excellent            85.5
Name: quiz_score, dtype: float64
```

The score histogram is centred around the middle bands, and the scatter plot shows that higher completion goes with higher scores (correlation 0.64). The bar chart shows the share of students who submitted the assignment against those who did not.

Insight and Business Value

A positive relationship between completion and scores would justify nudges that encourage students to finish videos. Early identification of low-engagement students enables timely mentoring, improving retention and course outcomes.

5. Public Transport Passenger Data Analysis

Scenario

A city transport department collects bus passenger records containing route number, stop, timestamp and passenger count. Planning better routes and reducing overcrowding requires clean, aggregated information about when and where demand is highest. The record-keeping process usually introduces invalid route codes, missing counts and duplicate entries, which must be corrected first.

Sample Dataset (Python)

The company's real data is not available, so a small synthetic dataset with the same kinds of defects is created below (random seed fixed for repeatability). All code in this section was executed and the results shown are the actual outputs.

Python code

```
import pandas as pd, numpy as np
pd.set_option('display.width', 90); pd.set_option('display.max_columns', 12)
np.random.seed(5)
n = 600
routes = ['R12', 'R21', 'R45', 'R88', 'R07']
stops = ['Central', 'T Nagar', 'Adyar', 'Guindy', 'Tambaram']
hour = np.random.choice(range(5, 23), n,
p=np.array([2,6,10,12,8,5,4,4,5,6,9,12,10,6,4,3,2,2]) / 110)
peak = np.isin(hour, [8, 9, 17, 18])
df = pd.DataFrame({'record_id': range(1, n + 1),
                   'route': np.random.choice(routes, n, p=[.3, .25, .2, .15, .1]),
                   'stop': np.random.choice(stops, n),
                   'hour': hour,
                   'passengers': (np.random.poisson(25, n) + peak * np.random.poisson(30, n))})
df.loc[np.random.choice(n, 4, replace=False), 'route'] = ['XX', 'R-0', '99Z', 'R00']    #
invalid routes
df.loc[np.random.choice(n, 25, replace=False), 'passengers'] = np.nan                  # missing
counts
df = pd.concat([df, df.sample(20, random_state=2)], ignore_index=True)                 #
duplicates
print(df.head())
print('Shape:', df.shape)
```

Output

```
   record_id route      stop  hour  passengers
0          1   R12    Guindy     8        48.0
1          2   R45     Adyar    18        51.0
2          3   R21    Guindy     8        54.0
3          4   R88  Tambaram    19        23.0
4          5   R12  Tambaram    13        25.0
Shape: (620, 5)
```

The dataset has 600 genuine boarding records with 4 invalid route codes, 25 missing passenger counts and 20 duplicated rows.

A. Use of Pandas

Pandas aggregation answers the department's questions directly. A time-slot column is derived from the timestamp (for example Morning Peak, Midday, Evening Peak and Night) using pd.cut() on the hour. groupby() then summarises passenger counts by route, stop and slot, and pivot_table() builds comparison grids.

- **By route:** df.groupby('route')['passengers'].sum() shows total demand per route.
- **By stop:** df.groupby('stop')['passengers'].mean() highlights the busiest stops.
- **By time slot:** df.groupby('time_slot')['passengers'].sum() shows demand through the day.
- **Combined:** df.pivot_table(index='route', columns='time_slot', values='passengers', aggfunc='sum').

Python code

```
df['time_slot'] = pd.cut(df['hour'], bins=[4, 10, 16, 20, 23],
                         labels=['Morning', 'Midday', 'Evening', 'Night'])
print('Total passengers by route:')
print(df.groupby('route')['passengers'].sum().sort_values(ascending=False))
print('\nAverage passengers per record by stop:')
print(df.groupby('stop')['passengers'].mean().round(1).sort_values(ascending=False))
print('\nRoute x time-slot table:')
print(df.pivot_table(index='route', columns='time_slot', values='passengers',
aggfunc='sum', observed=True))
```

Output

```
Total passengers by route:
route
R12    5944.0
R21    5652.0
R45    4696.0
R88    2841.0
R07    1983.0
XX       58.0
99Z      29.0
R-0      27.0
R00      25.0
Name: passengers, dtype: float64

Average passengers per record by stop:
stop
Guindy      37.4
Tambaram    37.2
Adyar       35.3
```

```
T Nagar     35.3
Central     33.8
Name: passengers, dtype: float64

Route x time-slot table:
time_slot  Morning  Midday  Evening  Night
route
99Z            NaN    29.0      NaN    NaN
R-0            NaN    27.0      NaN    NaN
R00            NaN     NaN       NaN   25.0
R07         1141.0   355.0    382.0  105.0
R12         2850.0  1279.0   1654.0  161.0
R21         2583.0  1372.0   1549.0  148.0
R45         1830.0  1163.0   1598.0  105.0
R88         1014.0   503.0   1266.0   58.0
XX            58.0     NaN      NaN    NaN
```

Grouping by route shows R12 carrying the most passengers (5,944), but invalid codes such as XX and 99Z also appear in the results, which proves that route validation is needed. The pivot table shows morning and evening as the heaviest slots.

B. Data Preprocessing

The cleaning operations needed for this data are summarised below.

Problem	Detection	Treatment
Invalid route numbers	Compare with a master route list: ~df['route'].isin(valid_routes)	Remove invalid rows, or correct obvious typing errors (extra spaces, lower case) using str.strip().str.upper() before validation.
Missing passenger counts	isnull().sum()	Fill with the median for the same route, stop and time slot; drop the row if no reasonable estimate exists.
Duplicate records	df.duplicated()	drop_duplicates() removes repeated entries caused by sensor or upload errors.

Python code

```
valid_routes = ['R12', 'R21', 'R45', 'R88', 'R07']
print('Invalid route records :', (~df['route'].isin(valid_routes)).sum())
df = df[df['route'].isin(valid_routes)]
print('Duplicate records     :', df.duplicated().sum())
df = df.drop_duplicates()
print('Missing passenger counts:', df['passengers'].isnull().sum())
df['passengers'] = df.groupby(['route', 'time_slot'], observed=True)['passengers'].transform(
    lambda s: s.fillna(s.median()))
df['passengers'] = df['passengers'].fillna(df['passengers'].median())
```

```
print('Missing after filling   :', df['passengers'].isnull().sum())
print('\nCleaned shape:', df.shape)
print(df.head())
```

Output

```
Invalid route records : 4
Duplicate records     : 20
Missing passenger counts: 25
Missing after filling   : 0

Cleaned shape: (596, 6)
   record_id route      stop  hour  passengers time_slot
0          1   R12    Guindy     8        48.0   Morning
1          2   R45     Adyar    18        51.0   Evening
2          3   R21    Guindy     8        54.0   Morning
3          4   R88  Tambaram    19        23.0   Evening
4          5   R12  Tambaram    13        25.0    Midday
```

Four invalid routes, 20 duplicates and 25 missing counts were handled. The cleaned table has 596 records and no missing passenger counts.

C. Visualization

Visual output identifies peak periods and congested corridors, supporting evidence-based scheduling.

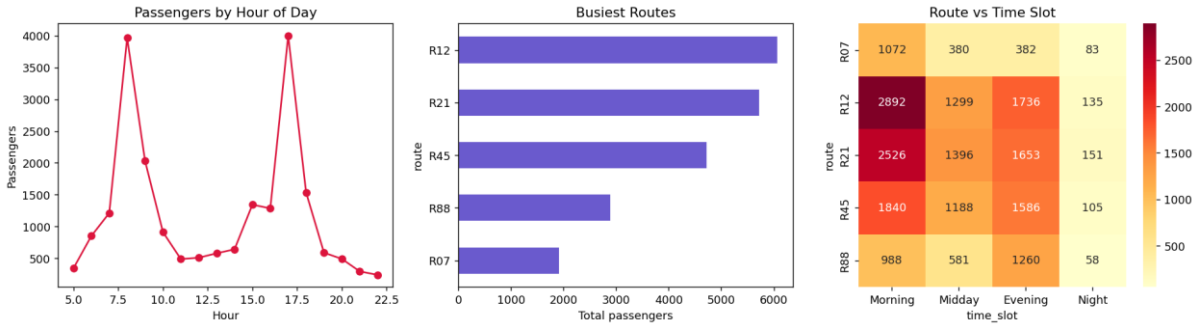
- **Line chart of passengers by hour –** shows morning and evening peak travel hours.
- **Horizontal bar chart of top routes –** ranks the busiest routes for additional buses.
- **Heatmap of route versus time slot –** shows passenger flow patterns and exactly where overcrowding occurs.
- **Box plot of load per stop –** shows variation and outlier stops with heavy boarding.

Python code

```
import seaborn as sns
import matplotlib.pyplot as plt
fig, ax = plt.subplots(1, 3, figsize=(15, 4.2))
df.groupby('hour')['passengers'].sum().plot(ax=ax[0], marker='o', color='crimson')
ax[0].set_title('Passengers by Hour of Day'); ax[0].set_xlabel('Hour');
ax[0].set_ylabel('Passengers')
df.groupby('route')['passengers'].sum().sort_values().plot(kind='barh', color='slateblue',
ax=ax[1])
ax[1].set_title('Busiest Routes'); ax[1].set_xlabel('Total passengers')
grid = df.pivot_table(index='route', columns='time_slot', values='passengers',
aggfunc='sum', observed=True)
sns.heatmap(grid, annot=True, fmt='.0f', cmap='YlOrRd', ax=ax[2]); ax[2].set_title('Route
vs Time Slot')
plt.tight_layout(); plt.show()
```

```
hourly = df.groupby('hour')['passengers'].sum()
print('Peak hour:', hourly.idxmax(), '- passengers:', int(hourly.max()))
print('Busiest route:', df.groupby('route')['passengers'].sum().idxmax())
```

Output – chart



Output – printed results

```
Peak hour: 17 - passengers: 4004
Busiest route: R12
```

Passenger volume peaks at 17:00 (4,004 passengers), with a smaller morning peak. R12 is the busiest route, and the heat map shows the evening slot on R12, R21 and R45 as the most crowded cells.

Insight and Business Value

Typical findings include sharp peaks around office hours and a handful of routes carrying a disproportionate share of passengers. The department can add buses or shorten headways on those routes at peak times, and reduce service where demand is low.

Conclusion and Summary of the Five Cases

The five case studies demonstrate that effective data science begins long before any chart is drawn. In every scenario, raw records were first explored and organised with Pandas, then repaired through targeted pre-processing, and finally communicated with Matplotlib and Seaborn. Skipping any stage reduces the trustworthiness of the results: duplicates inflate totals, unhandled missing values bias averages, unscaled variables distort comparisons, and poorly chosen charts hide patterns.

Summary of the Five Cases

Case	Key Pandas operations	Key preprocessing	Key visuals
1. Food delivery	read_csv, filter, datetime math	Impute delivery time, drop duplicates, fix prices	Histogram, top-restaurant bars, rating count plot
2. Loan approval	Column selection, loc, groupby, crosstab	Impute income, encode categories, scale loan amount	Box, violin and scatter plots by status
3. Energy monitoring	groupby, pivot_table, resample	IQR outlier removal, interpolation, totals	Bar chart per appliance, daily line chart
4. Online learning	sort_values, rank, nlargest	Impute scores, parse dates, pd.cut categories	Histogram, scatter, count plot
5. Public transport	groupby by route, stop, slot	Validate routes, impute counts, drop duplicates	Hourly line, route bars, heat map

Key Takeaways and References

- Always inspect data first with info(), describe() and isnull().sum() before choosing a cleaning strategy.
- Select the imputation technique according to the variable type, distribution and business meaning.
- Match the chart to the question: distributions need histograms or box plots, trends need line charts, categories need bar charts, and relationships need scatter plots or heat maps.
- Document every cleaning decision so that the analysis is reproducible and defensible.
- Interpret results with care, noting limitations such as imputation assumptions and sensitive attributes.

References

- McKinney, W. (2022). Python for Data Analysis, 3rd Edition. O'Reilly Media.
- VanderPlas, J. (2016). Python Data Science Handbook. O'Reilly Media.
- Pandas documentation: https://pandas.pydata.org/docs/
- Matplotlib documentation: https://matplotlib.org/stable/
- Seaborn documentation: https://seaborn.pydata.org/